DATABASE MANAGEMENT SYSTEMS LAB

Lab Report Submitted to

K. R. Mangalam University

for

Bachelor of Technology

in

Computer Science and Engineering

Submitted by

YUKTI VADEHRA, ROLL NO. 2401010128

Course Teacher

Ms. Mansi Kajal

School of Engineering & Technology

PRACTICAL-9

Aim

To implement **GRANT, REVOKE** commands and **SAVEPOINT** in SQL.

Consider the following tables namely “DEPARTMENTS” and “EMPLOYEES” Their schemas are as follows,
Departments (dept no , dept name , dept_location); Employees (emp_id , emp_name , emp_salary,dept_no);

a) Develop a query to grant all privileges of employees table into departments table
b) Develop a query to grant some privileges of employees table into departments table
c) Develop a query to revoke all privileges of employees table from departments table
d) Develop a query to revoke some privileges of employees table from departments table
e) Write a query to implement the save point.

Database Creation

```
CREATE DATABASE yukti_company_db;
USE yukti_company_db;
```

Table Creation

```
CREATE TABLE Departments (
  dept_no INT PRIMARY KEY,
  dept_name VARCHAR(50),
  dept_location VARCHAR(50)
);
CREATE TABLE Employees (
  emp_id INT PRIMARY KEY,
  emp_name VARCHAR(50),
  emp_salary DECIMAL(10,2),
  dept_no INT
);
```

Insert Values

```
INSERT INTO Departments VALUES
(101, 'HR', 'Delhi'),
(102, 'Finance', 'Mumbai'),
(103, 'IT', 'Bangalore');

INSERT INTO Employees VALUES
(1, 'Mansi', 50000, 101),
(2, 'Komal', 60000, 102),
(3, 'Sonal', 55000, 103),
```

```
(4, 'Neha', 65000, 101),
(5, 'Yukti', 70000, 102);
```

(a) Grant ALL privileges on Employees table

```
GRANT ALL PRIVILEGES
ON Employees
TO 'user1'@'localhost';
```

Explanation

- Gives full access (SELECT, INSERT, UPDATE, DELETE) on Employees table.

(b) Grant SOME privileges on Employees table

```
GRANT SELECT, INSERT
ON Employees
TO 'user1'@'localhost';
```

Explanation

- Allows only viewing and inserting data.

(c) Revoke ALL privileges

```
REVOKE ALL PRIVILEGES
ON Employees
FROM 'user1'@'localhost';
```

Explanation

- Removes all permissions from the user.

(d) Revoke SOME privileges

```
REVOKE INSERT
ON Employees
FROM 'user1'@'localhost';
```

Explanation

- Removes only INSERT permission.

(e) Implement SAVEPOINT

```
START TRANSACTION;
INSERT INTO Employees VALUES (6, 'Riya', 72000, 103);
SAVEPOINT sp1;
INSERT INTO Employees VALUES (7, 'Arjun', 75000, 101);
ROLLBACK TO sp1;
```

COMMIT;

16	18:32:29	START TRANSACTION	0 row(s) affected	0.000 sec
17	18:33:29	INSERT INTO Employees VALUES (6, 'Deepak', 72000, 103)	1 row(s) affected	0.000 sec
18	18:33:39	SAVEPOINT sp1	0 row(s) affected	0.000 sec
19	18:33:57	INSERT INTO Employees VALUES (7, 'Pulkit', 75000, 101)	1 row(s) affected	0.015 sec
20	18:34:09	ROLLBACK TO sp1	0 row(s) affected	0.000 sec
21	18:34:18	COMMIT	0 row(s) affected	0.016 sec

emp_id	emp_name	emp_salary	dept_no
1	Mansi	50000.00	101
2	Komal	60000.00	102
3	Sonal	55000.00	103
4	Neha	65000.00	101
5	Yukti	70000.00	102
6	Deepak	72000.00	103
NULL	NULL	NULL	NULL

Explanation

- SAVEPOINT creates a checkpoint.
- ROLLBACK TO undoes changes after savepoint.
- COMMIT saves final data.

Result

The database and tables were created successfully and records were inserted. The GRANT and REVOKE commands were used to assign and remove privileges on the Employees table. The SAVEPOINT command was implemented to control transactions and perform partial rollback, ensuring data consistency.

PRACTICAL- 10

Aim

To study and implement different types of SQL joins (**INNER JOIN, LEFT JOIN, RIGHT JOIN**) and subqueries to retrieve meaningful data from the *Employees* and *Departments* tables.

Database Creation

```
CREATE DATABASE yukti_company2_db;
USE yukti_company2_db;
```

Table Creation

```
CREATE TABLE Departments (
  dept_no INT PRIMARY KEY,
  dept_name VARCHAR(50),
  dept_location VARCHAR(50)
);
CREATE TABLE Employees (
  emp_id INT PRIMARY KEY,
  emp_name VARCHAR(50),
  emp_salary DECIMAL(10,2),
  dept_no INT
);
```

Insert Values

```
INSERT INTO Departments VALUES
(101, 'HR', 'Delhi'),
(102, 'Finance', 'Mumbai'),
(103, 'IT', 'Bangalore'),
(104, 'Marketing', 'Chennai');

INSERT INTO Employees VALUES
(1, 'Pulkit', 50000, 101),
(2, 'Neha', 60000, 102),
(3, 'Komal', 55000, 103),
(4, 'Sonal', 65000, 101),
(5, 'Yukti', 70000, 105);
```

(a) Display employee details where department matches

```
SELECT *
FROM Employees e
INNER JOIN Departments d
ON e.dept_no = d.dept_no;
```

	emp_id	emp_name	emp_salary	dept_no	dept_no	dept_name	dept_location
▶	1	Pulkit	50000.00	101	101	HR	Delhi
	2	Neha	60000.00	102	102	Finance	Mumbai
	3	Komal	55000.00	103	103	IT	Bangalore
	4	Sonal	65000.00	101	101	HR	Delhi

Explanation

- INNER JOIN returns only matching records
- Condition: same dept_no in both tables

(b) Employee name and Department name (LEFT JOIN)

```
SELECT e.emp_name, d.dept_name
FROM Employees e
LEFT JOIN Departments d
ON e.dept_no = d.dept_no;
```

	emp_name	dept_name
▶	Pulkit	HR
	Neha	Finance
	Komal	IT
	Sonal	HR
	Yukti	NULL

Explanation

- LEFT JOIN → all employees are shown
- If no matching department → NULL

(c) Employee name and Department name (RIGHT JOIN)

```
SELECT e.emp_name, d.dept_name
FROM Employees e
RIGHT JOIN Departments d
ON e.dept_no = d.dept_no;
```

	emp_name	dept_name
▶	Sonal	HR
	Pulkit	HR
	Neha	Finance
	Komal	IT
	NULL	Marketing

Explanation

- RIGHT JOIN → all departments shown
- If no employee → NULL

(d) Employees with salary greater than average

```
SELECT emp_name, emp_salary
FROM Employees
WHERE emp_salary > (
  SELECT AVG(emp_salary) FROM Employees
);
```

	emp_name	emp_salary
▶	Sonal	65000.00
	Yukti	70000.00

Explanation

- AVG() calculates average salary
- Subquery returns average
- Outer query selects employees with salary greater than average

Result

The tables were created and populated successfully. Various join operations were performed to retrieve matching and non-matching records between Employees and Departments tables. A subquery was used to identify employees earning more than the average salary, demonstrating effective use of SQL joins and aggregate functions.